VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Computer Networking

Assignment 1 Report

Weaprous Application

Instructor: Nguyen Phuong Duy

Group 1 - CC05

Name	**ID**
Tran Lam Bieu	2352116
Nguyen Anh Duy	2352181
Phan Dinh Cuong	2310402
Nguyen Manh Tri Minh	2352752
Dinh Cao Thien Loc	2352708

Ho Chi Minh City, 10/2025

Contents

1 System Overview **1**
1.1 Project Objectives . . . 1
1.2 Overall System Architecture . . . 1
1.2.1 Server-Side Processes . . . 1
1.2.2 Client-Side Peer Process . . . 2

2 Task 1: Build HTTP Server with cookie session **2**
2.1 Task 1 Objective . . . 2
2.2 Basic HTTP Handling . . . 3
2.2.1 The Role and Workflow of `HttpAdapter` . . . 3
2.2.2 Role . . . 3
2.2.3 Workflow (`handle_client` Core Flow) . . . 3
2.2.4 Request: parse headers & cookies . . . 4
2.2.5 Response: Building Header/Body & Serving Static/JSON . . . 5
2.3 Authentication and Session Management . . . 6
2.3.1 Task 1A: Login Procedure . . . 6
2.3.2 Task 1B: Cookie/Session & Resource Protection . . . 7
2.4 Reverse proxy: configuration and Processing . . . 7
2.4.1 Round-robin configuration . . . 7
2.4.2 Handle Per Connection . . . 7
2.4.3 Round-robin logic . . . 8

3 Task 2: Hybrid P2P Chat Application **9**
3.1 Objective . . . 9
3.2 System Design . . . 9
3.2.1 Tracker (WebApp) . . . 9
3.2.2 Peer Node . . . 9
3.3 Implementation Details . . . 10
3.3.1 Console Client `chatAppP2P.py` . . . 10
3.3.2 TUI Client `chatAppP2PUI.py` . . . 11
3.4 Testing and Evaluation . . . 12
3.5 Summary of Task 2 . . . 12

1 System Overview

1.1 Project Objectives

The primary objective of this project is to engineer a network architecture based on a **hybrid model**, designed to function simultaneously as a traditional client–server framework and as a platform supporting direct peer-to-peer (P2P) communication between user nodes.

Our development methodology follows a bottom-up approach, structured into distinct phases:

1. **Phase 1: HTTP Server Implementation.** Implementation of a foundational HTTP server utilizing raw TCP sockets. This server manually handles request parsing and response construction, incorporating user authentication and session management mechanisms via cookies (addressing **Task 1A** and **Task 1B**).

2. **Phase 2: Hybrid Chat Application Deployment.** Leveraging this server infrastructure to deploy a hybrid chat application. The operation is divided into two stages:

   - **Discovery Phase (Client–Server):** Peers register their presence with a centralized server and retrieve a directory of other active peers.
   - **Communication Phase (P2P):** Peers establish direct TCP socket connections to exchange messages, bypassing the central server entirely.

In summary, the final deliverables include: (1) a custom-built, minimalist HTTP server featuring login and cookie-based access control, and (2) a hybrid chat application that utilizes the client–server model for initialization and the P2P model for message transmission.

1.2 Overall System Architecture

The overall system architecture is composed of three distinct server-side processes and multiple client-side peer processes:

1.2.1 Server-Side Processes

1. **Proxy Process** The Proxy acts as the centralized gateway (defaulting to port 8080) for all incoming client traffic. It parses the configuration from `config/proxy.conf`, extracts the `Host` from HTTP requests, and performs reverse proxy routing to the designated backend. Where a single Host maps to multiple backends, the Proxy implements a **round-robin** load balancing strategy.

2. **Backend Process** The Backend is tasked with processing raw HTTP packets received via TCP. It decodes the method, path, headers, and body to generate appropriate HTTP responses. For Task 1, the backend executes the logic for user authentication, including the issuance and validation of cookies for accessing protected resources.

3. **WebApp Process (WeApRous)** The WebApp serves as a lightweight framework providing RESTful API endpoints (supporting `GET`, `POST`, `PUT`, `DELETE` methods). In the context of Task 2, the WebApp functions as a **"tracker,"** maintaining a registry of online peers and distributing this list to requesting clients.

1.2.2 Client-Side Peer Process

On the user side, each **peer process** operates independently and fulfills a dual role:

- **As an HTTP Client:** It connects to the WebApp to register credentials (name, IP, port) and fetch the list of active peers.
- **As a TCP Server:** It listens on a socket to accept incoming connections, facilitating direct P2P chat sessions.

The operational data flow of the system is structured as follows:

- **Client → Proxy → Backend:** Handles standard HTTP interactions, authentication, and session management (Task 1A, 1B). The Proxy manages routing and load balancing across backends.
- **Backend/WebApp (WeApRous):** Exposes RESTful endpoints for peer registration and discovery.
- **Peer Initialization:** The peer retrieves the active peer list from the WebApp and subsequently initiates direct TCP connections with other users.
- **Peer ↔ Peer:** Facilitates decentralized, direct message exchange.

2 Task 1: Build HTTP Server with cookie session

2.1 Task 1 Objective

The core objective of **Task 1** (encompassing Task 1A and Task 1B) is the construction of a fundamental **HTTP Server** from the ground up, utilizing raw TCP socket programming. This server must fulfill several crucial functions:

1. **Basic HTTP Handling:** Manually parse incoming HTTP requests and construct valid HTTP responses.
2. **Authentication and Session Management:** Implement user authentication via a login mechanism and manage the user's login state using **HTTP Cookies**.
3. **Load Balancing:** Ensure the system architecture supports distributing client traffic across multiple Backend instances for a given Host, utilizing the **Round-Robin** policy as defined in the Proxy configuration (`config/proxy.conf`)

2.2 Basic HTTP Handling

2.2.1 The Role and Workflow of `HttpAdapter`

`HttpAdapter` serves as the crucial "middleman" between the raw TCP socket communication and the high-level HTTP processing and application logic.

2.2.2 Role

The `HttpAdapter` acts as the interface between the networking layer (sockets) and the HTTP business logic:

1. **Data Reception:** Handles receiving raw data from the socket (`recv`).
2. **Request Parsing:** Delegates the parsing of the raw request data using `Request.prepare()` to obtain the method, path, headers, body, and cookies.
3. **Route Decision:** Determines the routing decision: if a `hook/handler` (REST) is matched, it calls the handler; otherwise, it serves static content or default logic (login/cookie).
4. **Response Transmission:** Builds the final response using `Response` and transmits it back via the socket.

2.2.3 Workflow (`handle_client` Core Flow)

The `handle_client` method outlines the core flow for processing a single incoming HTTP request:

```
    def handle_client(self, conn, addr, routes):
        # Connection handler.
        self.conn = conn
        # Connection address.
        self.connaddr = addr
        # Request handler
        req = self.request
        # Response handler
        resp = self.response
        # Handle the request
        msg = conn.recv(1024).decode()
        req.prepare(msg, routes)

        response = None

        # Handle request hook
        if req.hook:
            hook_response = req.hook(headers = req.headers, body = req.body)
            response = resp.build_response(req, hook_response)

        else:
            if req.method == POST and req.path == /login:
                if req.body.get(username) == admin and req.body.get(password) ==
password:
                    req.path = /index.html
                    resp.headers['Set-Cookie'] = 'auth=true;␣HttpOnly;␣Path=/'
```

```
                    response = resp.build_response(req)
                else:
                    response = resp.build_unauthorized()
            elif req.method == GET:
                if req.path == / or req.path == /index.html:
                    if req.cookies and req.cookies.get(auth) == true:
                        req.path = /index.html
                    else:
                        req.path = /login.html
                response = resp.build_response(req)
        if response:
            conn.sendall(response)
        conn.close()
```

Listing 1: daemon/httpadapter.py

- **Parsing (`req.prepare`):** Parses the raw request data (`msg`) and checks for a dynamic route match, assigning it to `req.hook`.
- **Dynamic Routing:** If `req.hook` is present, the function is executed to generate `hook_response`.
- **Authentication (Task 1A):** If `POST /login` is received, credentials are validated. Success leads to setting the `Set-Cookie` header and serving `/index.html`. Failure results in an unauthorized response.
- **Access Control (Task 1B):** If `GET /` or `/index.html` is requested, the presence of the `auth=true` cookie is checked. Access is granted (`req.path = "/index.html"`) or redirected to the login page (`req.path = "/login.html"`) based on the cookie status.

2.2.4 Request: parse headers & cookies

The `Request` class is responsible for converting the raw data stream received from the socket into structured HTTP components:

- Separating the `start-line` (comprising `METHOD`, `PATH`, and `VERSION`).
- Normalizing `headers` into a `CaseInsensitiveDict`.
- Parsing the `body` (specifically `form/urlencoded`).
- Extracting `Cookie` pairs into a dictionary (`dict`) for backend validation.

```
    def prepare_headers(self, request):
        lines = request.split('\r\n')
        headers = {}
        for line in lines[1:]:
            if ':␣' in line:
                key, val = line.split(':␣', 1)
                headers[key.lower()] = val
        return headers

```

```python
    def prepare_cookies(self):
        cookies_string = self.headers.get('cookie', '')
        self.cookies = {}  # Initialize cookie dict
        if cookies_string:
            pairs = cookies_string.split(';')
            for pair in pairs:
                pair = pair.strip()
                if '=' in pair:
                    # Split only on the first '='
                    key, val = pair.split('=', 1)
                    self.cookies[key.strip()] = val.strip()

    def prepare(self, request, routes=None):
        # Prepare the request line from the request header
        self.method, self.path, self.version = self.extract_request_line(request)

        if not routes == {}:
            self.routes = routes
            self.hook = routes.get((self.method, self.path))

        self.headers = self.prepare_headers(request)

        self.prepare_cookies()
        self.prepare_body(request, None, json)
        return
```

Listing 2: daemon/request.py

2.2.5 Response: Building Header/Body & Serving Static/JSON

The `Response` class is responsible for assembling the `status line`, `headers` (including `Set-Cookie` when necessary), and `body` into the final HTTP response. The central function, `build_response()`, determines the content serving strategy based on the MIME type:

- **HTML/CSS/Images (Static):** Selects the appropriate `base_dir` and reads the file content in binary mode.
- **JSON API:** If `json_data` is present, it sets `Content-Type: application/json` and encodes the data using `UTF-8`.
- **Error Handling:** If no type is matched and `json_data` is absent, it returns a `404 Not Found` response.

```python
    def build_response(self, request):
        path = request.path
        mime_type = self.get_mime_type(path)
        base_dir =

        #If HTML, parse and serve embedded objects
        if path.endswith('.html') or mime_type == 'text/html':
            base_dir = self.prepare_content_type(mime_type = 'text/html')
        elif mime_type == 'text/css':
            base_dir = self.prepare_content_type(mime_type = 'text/css')
```

```
        elif mime_type.startswith('image/'):
            base_dir = self.prepare_content_type(mime_type=mime_type)
        elif mime_type.startswith('video/'):
            base_dir = self.prepare_content_type(mime_type=mime_type)
        elif mime_type in ('application/javascript', 'application/x-javascript'):
            base_dir = self.prepare_content_type(mime_type='application/javascript')
        elif mime_type in ('application/json', 'text/xml', 'application/xml', 'text/
    csv', 'application/zip'):
            base_dir = self.prepare_content_type(mime_type=mime_type)
        else:
            return self.build_notfound()
        if self.status_code is None:
            self.status_code = 200
        self.reason = OK
        c_len, self._content = self.build_content(path, base_dir)
        self._header = self.build_response_header(request)

        return self._header + self._content
```

Listing 3: daemon/response.py: buildresponse

2.3 Authentication and Session Management

2.3.1 Task 1A: Login Procedure

This task outlines the process for user authentication, from the client request to the backend validation and cookie issuance.

1. **Client Request:** The Client sends a `POST /login` request through the Proxy (with Host 8080) containing the `username` and `password` in the request body.

2. **Proxy Routing:** The Proxy reads the `Host` header, selects a Backend based on the configured routes (potentially using a `round-robin` load balancing policy), and forwards the raw request.

3. **Backend Parsing:** The Backend process receives the raw bytes via TCP, and the `HttpAdapter` parses the data into a structured `Request` object.

4. **Authentication Logic (Adapter):** The `HttpAdapter` recognizes the `POST /login` endpoint:

   - **Success:** If the login information is correct, it sets the session cookie: `resp.headers["Set-Cookie"] = "auth=true; HttpOnly; Path=/"`; changes the internal request path to `/index.html`; and builds a `200 OK` response.
   - **Failure:** If the information is incorrect, it builds a `401 Unauthorized` response.

5. **Final Response:** The Proxy receives the response from the Backend and transmits it back to the Client.

2.3.2 Task 1B: Cookie/Session & Resource Protection

This task details how the server uses the issued cookie to enforce access control over protected resources.

1. **Protected Request:** The Client proceeds to send a `GET /` (or `/index.html`) request through the Proxy. The browser automatically attaches the session cookie: `Cookie: auth=true`.
2. **Proxy Forwarding:** The Proxy forwards the request using the same mechanism as before.
3. **Access Control (Backend):** The Backend parses the request and checks the value of `req.cookies.get("auth")`:
   - **Authenticated:** If `auth=true`, it grants access to `index.html` (the "protected" content) and serves the file.
   - **Unauthenticated:** If the cookie is missing or incorrect, it redirects the internal path to `/login.html` or returns a `401 Unauthorized` response.

2.4 Reverse proxy: configuration and Processing

2.4.1 Round-robin configuration

The Proxy process is responsible for receiving all incoming client requests, determining the correct backend target based on the request's `Host` header, and forwarding the request.

```
host localhost:8080 {
    proxy_pass http://127.0.0.1:9001;
    proxy_pass http://127.0.0.1:9002;
    proxy_pass http://127.0.0.1:9003;
    dist_policy round-robin }
```

Listing 4: Example of mapping and round-robin configuration

2.4.2 Handle Per Connection

The handle$_{c}lientfunctionmanagestherequestlifecyclewithintheProxy$: $extraction, routing, forwarding, andresponding.$

```

def handle_client(ip, port, conn, addr, routes):
    request = conn.recv(1024).decode()

    # Extract hostname
    for line in request.splitlines():
        if line.lower().startswith('host:'):
            hostname = line.split(':', 1)[1].strip()

    # Resolve the matching destination in routes and need conver port
    # to integer value
    resolved_host, resolved_port = resolve_routing_policy(hostname, routes)
    try:
```

```
        resolved_port = int(resolved_port)
    except ValueError:
        print(Not␣a␣valid␣integer)
    if resolved_host:
        response = forward_request(resolved_host, resolved_port, request)
    else:
        response = (
            HTTP/1.1␣404␣Not␣Found\r\n
            Content-Type:␣text/plain\r\n
            Content-Length:␣13\r\n
            Connection:␣close\r\n
            \r\n
            404␣Not␣Found
        ).encode('utf-8')
    conn.sendall(response)
    conn.close()
```

Listing 5: Handler at proxy

2.4.3 Round-robin logic

The Round-Robin logic ensures that traffic for a specific hostname is distributed sequentially and cyclically across its mapped backends. The mechanism uses a dictionary _rr_idx to track the current index for each unique hostname, thus ensuring independent rotation for each configured virtual host.

```
def resolve_routing_policy(hostname, routes):
    proxy_map, policy = routes.get(hostname,('127.0.0.1:9000','round-robin'))
    proxy_host = ''
    proxy_port = '9000'
    if isinstance(proxy_map, list):
        if len(proxy_map) == 0:
            proxy_host = '127.0.0.1'
            proxy_port = '9000'
        elif len(proxy_map) == 1:
            proxy_host, proxy_port = proxy_map[0].split(:, 2)
        elif len(proxy_map) >= 2: # apply the policy handling
            if policy == round-robin:
                with _rr_lock:
                    current = _rr_idx.get(hostname, 0)
                    i = current % len(proxy_map)
                    _rr_idx[hostname] = current + 1

                target = proxy_map[i]

            else:
                target = proxy_map[0]

            proxy_host, proxy_port = target.rsplit(:, 1)
        else:
            # Out-of-handle mapped host
            proxy_host = '127.0.0.1'
            proxy_port = '9000'
    else:
        proxy_host, proxy_port = proxy_map.split(:, 2)
```

```

    return proxy_host, proxy_port
```

Listing 6: Handler at proxy

3 Task 2: Hybrid P2P Chat Application

3.1 Objective

Task 2 extends the system from a simple HTTP server (Task 1) to a **hybrid chat application**:

- The **WebApp (tracker)** works in client–server mode to register peers and provide a list of online nodes.
- The chat itself uses **direct P2P TCP connections** between peers, with **channel**-based messaging.

We implemented two clients that share the same protocol:

- A console client: `chatAppP2P.py`.
- A Textual-based TUI client: `chatAppP2PUI.py`.

3.2 System Design

3.2.1 Tracker (WebApp)

The WebApp plays the role of a **tracker** that maintains the list of active peers. It exposes simple REST endpoints:

- `POST /submit-info`: register or update a peer with fields `name`, `ip`, `port`.
- `GET /get-list`: return the current online peer list in JSON.
- `DELETE /delete-info`: remove a peer when it logs out or exits.

Internally, the tracker stores peer entries in an in-memory data structure (e.g., dictionary keyed by peer name). It overwrites entries on re-registration and deletes them on logout.

3.2.2 Peer Node

Each peer node has two roles:

- **HTTP client** to the tracker, using the `requests` library.
- **P2P node** running a TCP server and connecting to other peers.

Peers agree on a simple JSON format:

- Initial handshake (identification):

```
{"name": "<peer_name>", "ip": "<peer_ip>", "port": <peer_port>}
```

- Chat message:

```
{"channel": "<channel_name>",
 "from": "<sender_name>",
 "message": "<text>"}
```

Channel filtering is done locally: a peer only prints messages whose `"channel"` matches its current channel.

3.3 Implementation Details

3.3.1 Console Client `chatAppP2P.py`

3.3.1.1 Global configuration.

The client keeps global state such as:

- `MY_IP`, `MY_PORT`, `MY_NAME`.
- `SERVER_URL` (tracker HTTP endpoint).
- `peers`: list returned from `/get-list`.
- `connected_peers`: currently established P2P connections.
- `current_channel`, `known_channels`.

A `threading.Lock` protects access to `connected_peers`.

3.3.1.2 Tracker interaction.

- `do_login(name)`: sends `POST /submit-info` with the peer info and sets `LOGGED_IN = True`.
- `unregister_peer()`: sends `DELETE /delete-info` before exit.
- `heartbeat_loop()`: background thread that re-submits `/submit-info` every few seconds to keep the tracker updated.

3.3.1.3 P2P server.

A dedicated thread runs:

```python
def server_thread(ip, port):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.bind((ip, port))
    s.listen(5)
    while True:
        conn, addr = s.accept()
        threading.Thread(target=handle_receive, args=(conn,)).start()
```

Listing 7: P2P server thread (simplified)

In `handle_receive`, the peer:

1. Reads the handshake JSON and adds the connection to `connected_peers`.
2. Continuously receives JSON messages.
3. Displays messages only if the `channel` matches `current_channel`.

3.3.1.4 P2P client and broadcast.

- `fetch_peer_list()`: calls `GET /get-list` and populates `peers`.
- `connect_to_peer(peer)`: creates a TCP connection, sends handshake JSON, then starts a receive thread.

When the user types a normal line (not a command), the client creates a JSON payload and sends it to every socket in `connected_peers`:

```
payload = json.dumps({
    channel: current_channel,
    from: MY_NAME,
    message: text,
})
for p in connected_peers:
    p[socket].send(payload.encode())
```

Listing 8: Broadcasting a message

3.3.1.5 User commands.

The console supports commands such as:

- `/login <name>`: register to tracker.
- `/join <channel>`: change current channel.
- `/channels`, `/peers`, `/refresh`.
- `/logout`, `/exit`.

3.3.2 TUI Client `chatAppP2PUI.py`

The TUI client uses the same networking functions but wraps them inside a `textual.app.App`.

3.3.2.1 Layout.

The screen is divided into:

- A **chat log** panel for messages and system events.
- A **peers** panel listing connected peers.
- A **channels** panel showing available channels and the current one.
- An **input** box for commands/messages and a small status bar.

3.3.2.2 Networking.

The TUI calls the same helper functions:

- Starts a P2P server thread.
- Registers to the tracker and launches the heartbeat thread.
- Periodically refreshes the peer list and connects to new peers.

Broadcast and receive logic is analogous to the console client, but output goes to UI widgets instead of `stdout`.

3.4 Testing and Evaluation

To test Task 2, we followed these steps:

1. Run the tracker (WebApp) on `127.0.0.1:8000`.
2. Start multiple peers on different ports (both console and TUI versions).
3. Use `/login` and `/refresh` so each peer obtains the peer list and opens P2P connections.
4. Join a common channel (e.g., `/join general`) and exchange messages.

Observed behaviour:

- All peers correctly appeared in the tracker list.
- Messages were delivered in real time between connected peers.
- Channel filtering worked: messages on channel `general` did not appear in other channels.
- When a peer exited, other peers eventually removed it from their peer lists after connection errors.

3.5 Summary of Task 2

Task 2 successfully demonstrates a simple but complete **hybrid chat system**: centralized tracker for peer discovery and fully decentralized TCP communication for the chat itself, with channel-based message routing and both CLI and TUI interfaces built on top of the same protocol.